Making Model Runs Auditable by Default: A Design Science Study of Signed, Replayable Capability Receipts for Organizational AI Accountability

Lakshman Turlapati[1]
Rabih Neouchi[2]

[1]Full Self Browsing

[2]Naveen Jindal School of Management, The University of Texas at Dallas, Richardson, TX, USA

Corresponding author: Lakshman Turlapati, alumnus, Naveen Jindal School of Management, The University of Texas at Dallas (M.S.,

Abstract

Organizations embed large language models in business processes, yet auditors need evidence of what a system was asked, how it was configured, and what it produced. This design-science study states five requirements and four principles for run-level accountability and instantiates them in Lattice, an open-source runtime. When a signer is configured, a run emits a canonical receipt that is redacted, signed, and replayable. An offline evaluation on mocked providers rejects tampering of single receipts, replays recorded runs to the signed output hash, and adds milliseconds of signing cost. Replay binding, a version floor, and open-chain length checks stay verifier opt-ins. New crew runs close the receipt chain. Personal data matched by a privacy check is masked before signing. The study contributes design knowledge for accountable AI information systems.

Keywords: AI accountability; Design science research; Audit trail; Large language models; Provenance; AI governance

1 Introduction

Generative artificial intelligence (AI) has moved from experimentation into operational information systems that approve refunds, triage claims, summarize case files, and draft regulated communications (Berente et al., 2021; Bommasani et al., 2021). These systems combine text, images, documents, and tool calls, and they often route a single task across several model providers for reasons of cost, latency, privacy, or capability (Chen et al., 2023; Ong et al., 2024). Their outputs therefore carry organizational consequences, and with consequences comes accountability: someone must be able to explain, after the fact, what the system was asked to do, which model served the request under which constraints, and what came back (Wieringa, 2020; Novelli et al., 2023).

Regulatory and standards frameworks now make this expectation explicit. The European Union Artificial Intelligence Act requires providers of high-risk systems to enable automatic recording of events over the

system's lifetime and to keep such logs (European Parliament and Council of the European Union, 2024). The NIST AI Risk Management Framework asks organizations to document and monitor AI systems so that risks can be measured and managed (Tabassi, 2023), and ISO/IEC 42001 specifies a management system in which AI-related processes must be controlled and evidenced (ISO/IEC, 2023). Scholarship on AI auditing reaches a similar conclusion: internal and external audits depend on artifacts that make claims about a system verifiable rather than merely asserted (Raji et al., 2020; Brundage et al., 2020; Mökander and Floridi, 2021; Mökander et al., 2023).

Current practice falls short of this. Most organizations integrate models through provider wrappers and orchestration libraries that normalize request formats and add conveniences such as tool calling and tracing (Amershi et al., 2019; Sculley et al., 2015). Their traces are written for human debugging. A trace can be edited after the fact, it does not bind inputs to outputs, and a third party cannot check it without trusting the party that produced it (Schneier and Kelsey, 1999; Crosby and Wallach, 2009). The result is an accountability gap between what governance frameworks expect and what AI-enabled information systems actually record (Raji et al., 2020; Papagiannidis et al., 2025).

This study addresses that gap through design science research (DSR) (Hevner et al., 2004; Peffers et al., 2007; Gregor and Hevner, 2013). The research question is: *How should a model-run infrastructure be designed so that every run of an AI-enabled information system yields evidence that auditors can verify independently and operators can reproduce, at acceptable cost?* We derive design requirements from the governance and auditing literature, formulate design principles, instantiate them in an open-source artifact called Lattice (Turlapati, 2026b), and evaluate the artifact against each requirement with a reproducible, offline evaluation harness. The central construct is the *capability receipt*: a per-run record that is canonicalized, redacted before signing, signed with a public-key signature, and linked to content-addressed artifacts so that it can be verified and replayed offline.

The study makes three contributions. First, it specifies requirements and principles for run-level AI accountability, a nascent design theory in the sense of Gregor and Jones (2007) that is independent of any particular implementation. Second, it provides an instantiation whose behavior is evaluated quantitatively, covering integrity under attack, replay fidelity, policy enforcement, and runtime overhead. Third, it reports where the instantiation does *not* meet its requirements, findings that are themselves design knowledge for builders of accountable AI systems (Baskerville et al., 2018). We position the contribution as an *improvement* in the sense of Gregor and Hevner (2013): a known problem, auditable record keeping, addressed with a new solution for a new class of systems, multi-provider model runs.

The remainder of the paper follows the DSR publication schema proposed by Gregor and Hevner (2013). Section 2 reviews related work. Section 3 describes the research approach. Section 4 derives requirements and principles. Section 5 presents the artifact. Section 6 reports the evaluation. Section 7 discusses contributions, implications, and limitations, and Section 8 concludes.

2 Background and Related Work

2.1 AI accountability, governance, and auditing

Accountability in AI denotes the obligation of an actor to explain and justify the conduct of a system to a forum that can pass judgment and impose consequences (Wieringa, 2020; Novelli et al., 2023). IS research frames AI management as a matter of autonomy, learning, and inscrutability (Berente et al., 2021), and it describes how organizations envelop AI with sociotechnical controls to contain its behavior (Asatiani et al., 2021). Organizational AI governance comprises the structures, processes, and procedures that align AI use with organizational and regulatory goals (Mäntymäki et al., 2022; Schneider et al., 2022; Papagiannidis et al., 2025). Across these accounts, governance depends on evidence. Policies and controls mean little unless an organization can show, for specific decisions, that the controls operated.

The AI auditing literature makes this dependence concrete. Raji et al. (2020) propose an end-to-end internal audit framework and stress documentation artifacts throughout the development life cycle. Mökander and Floridi (2021) argue for ethics-based auditing as a governance mechanism, and Mökander et al. (2023) propose a three-layered approach to auditing large language models at the governance, model, and application layers. Brundage et al. (2020) call for mechanisms, including audit trails, that make claims about AI development verifiable by outsiders. Transparency research cautions that transparency must be designed for specific audiences and purposes (Felzmann et al., 2019; Rai, 2019), and human-centered AI calls for reliable, safe, and trustworthy systems backed by audit trails (Shneiderman, 2020). Documentation instruments such as model cards (Mitchell et al., 2019) and datasheets for datasets (Gebru et al., 2021) standardize disclosure about models and data. However, they describe artifacts at release time, not individual production runs. The application layer of Mökander et al. (2023), where a model is used in a specific business process, is where the evidence gap is widest, and it is the layer this study targets.

2.2 Tamper-evident logging, provenance, and transparency

Computer security offers a long tradition of tamper-evident records. Haber and Stornetta (1991) showed how to time-stamp digital documents, and Merkle (1988) introduced hash-tree commitments. Schneier and Kelsey (1999) designed secure audit logs whose entries cannot be undetectably altered after compromise, and Crosby and Wallach (2009) gave efficient data structures for tamper-evident logs with logarithmic proofs. Certificate Transparency (Laurie et al., 2013) publishes certificates to append-only logs so that misissuance becomes publicly detectable. Software supply chain research applies these ideas to builds. in-toto (Torres-Arias et al., 2019) signs each step of a software supply chain, The Update Framework addresses survivable key compromise (Samuel et al., 2010), Sigstore makes signing and transparency logging accessible to developers (Newman et al., 2022), and SLSA defines levels of supply chain integrity (OpenSSF, 2023). The Dead Simple Signing Envelope (DSSE) specifies a minimal signed envelope with a pre-authentication encoding (Secure Systems Lab, 2021). Data provenance research models the derivation history of data products (Moreau et al., 2011; Herschel et al., 2017).

These lines of work supply mechanisms (canonical encodings, signatures, hash chains, transparency logs) but were not designed for the semantics of a model run. A model run has a declared intent, a routing

decision among providers, policy constraints, output invariants, and multimodal inputs that may contain personal data.

2.3 Verifiable inference and model routing

A complementary stream seeks cryptographic guarantees that a specific model computed a specific output, using zero-knowledge proofs (Kang et al., 2022; Sun et al., 2024). Such proofs give strong guarantees but currently require access to model weights and impose high costs, so they do not apply to proprietary hosted models consumed through application programming interfaces (APIs). Work on model routing optimizes cost and quality by selecting among models (Chen et al., 2023; Ong et al., 2024), often with learned routers whose decisions are hard to audit. Security research on LLM-integrated applications shows that model outputs can be manipulated through indirect prompt injection (Greshake et al., 2023), which raises the stakes for run-level evidence.

2.4 Agent receipts and verifiable agent provenance

Three recent proposals are closest to this study. *TrustTrack* is a vision paper that argues for verifiable identity, policy commitments, and tamper-resistant behavioral logs in multi-agent systems (Li, 2025). *Sello* ("Notarized Agents") inverts the trust boundary: the service that receives an agent's call signs a receipt, encrypts it to the agent's owner, and publishes it to a witness-cosigned Merkle log, so that the owner need not trust the agent or its operator (Figuera, 2026). *Janus* places a signed, hash-chained log on the effect path of agentic LLMs, so that a step's proposal and verdict are durable before its effect is released. Its gates are pure functions of the log and can be re-derived offline, and it is evaluated under crash injection and with a lending workflow (Arslan, 2026).

Table 1 compares these systems with the artifact presented here. The comparison shows that the contribution of this study is not the idea of signed records for AI, which all four share, but a specific combination of design features aimed at *organizational* model use: issuer-side receipts emitted for every run that has a signer, a recorded and deterministic routing decision across heterogeneous providers, redaction before signing, and offline replay. The comparison also shows what Lattice lacks: Sello's receiver-side attestation and transparency log, and Janus's effect gating and crash-consistency evaluation. Lattice was designed independently. Its public repository dates from 21 April 2026 (Turlapati, 2026b), before the Sello (June 2026) and Janus (September 2026) preprints appeared.

3 Research Approach

We follow the design science research methodology (DSRM) of Peffers et al. (2007), guided by the DSR guidelines of Hevner et al. (2004) and the contribution framework of Gregor and Hevner (2013). Table 2 maps the six DSRM activities to this study. The entry point is problem-centered: the accountability gap arose while building a browser-automation product whose autonomous actions required an auditable record, and it generalizes to any organization that embeds models in business processes. DSR builds and evaluates artifacts to solve organizational problems while producing prescriptive knowledge (vom Brocke et al., 2020),

Table 1: Comparison of run-level and agent-level accountability approaches (based on each system's published description)

Feature	TrustTrack	Sello	Janus	Lattice (this study)
Unit of record	Agent behavior	Agent action at receiving service	Agent step (proposal, verdict, effect)	Model run (intent, route, outputs)
Who signs	Protocol participants (design)	Receiving service	Agent runtime, validators, humans	Issuing runtime (self-attested)
Tamper evidence	Proposed	Signature and witness-cosigned Merkle log	Signed hash chain	Signature over canonical JSON; optional parent links
Third-party transparency log	Not specified	Yes	No	No
Deterministic, recorded routing across providers	No	No	No	Yes
Redaction before signing	Not specified	Encryption to owner	Not a focus	Yes, masked before signing (Sect. 6.4)
Offline replay	Not specified	Owner reconstructs trail	Re-derive verdicts offline	Re-materialize run and compare output hash
Effect gating	No	No	Yes	Terminal output invariants (no retry)
Implementation	None (blueprint)	Reference implementation	Implementation	Open-source library and command-line tool
Evaluation	Use cases	Cryptographic microbenchmarks	Crash injection, 100M-event verification, model experiment	Tamper suite, replay, policy checks, overhead (Sect. 6)

Table 2: Mapping of DSRM activities (Peffers et al., 2007) to this study

DSRM activity	This study
1. Problem identification	Accountability gap between governance expectations and model-run records (Sect. 1, 2)
2. Objectives of a solution	Design requirements DR1–DR5 (Sect. 4)
3. Design and development	Design principles DP1–DP4 instantiated in Lattice (Sect. 5)
4. Demonstration	Runnable scenarios: a multimodal work inbox, a single-agent loop, and a multi-agent crew
5. Evaluation	Offline, artificial evaluation against DR1–DR5; planned practitioner study (Sect. 6)
6. Communication	This article; open-source artifact and evaluation harness

and it has long been linked to information systems design theories (Walls et al., 1992). We use the anatomy of a design theory (Gregor and Jones, 2007) to structure the knowledge contribution, with requirements as meta-requirements, principles as principles of form and function, and Lattice as an expository instantiation. Evaluation follows the Framework for Evaluation in Design Science (FEDS) of Venable et al. (2016). We adopt a technical risk and efficacy strategy, starting with artificial, summative evaluation of the instantiation against its requirements, and plan a naturalistic evaluation with practitioners as the next episode (Section 6.7). Evaluation criteria are drawn from the taxonomy of Prat et al. (2015) (goal attainment, efficiency, robustness, and consistency) and from the build-evaluate pattern of Sonnenberg and vom Brocke (2012).

4 Design Requirements and Principles

4.1 Design requirements

We derive five design requirements (DRs) from the governance, auditing, and secure logging literature. Each is stated at the level of the class of systems rather than of any particular implementation.

DR1 (Integrity and attribution). The record of each model run must be tamper-evident and attributable to an accountable issuer, so that any alteration is detectable by a party holding only public verification material. This follows from logging duties for high-risk systems (European Parliament and Council of the European Union, 2024), from the verifiable-claims agenda (Brundage et al., 2020), and from secure audit log research (Schneier and Kelsey, 1999; Crosby and Wallach, 2009).

DR2 (Reproducibility). An auditor must be able to reconstruct a recorded run offline and confirm that the recorded inputs and outputs are those committed in the record, without access to the original system or provider. This reflects the role of reproducibility in audits (Raji et al., 2020) and in ML practice more broadly (Kapoor and Narayanan, 2023).

DR3 (Inspectable decisions and enforced policy). The record must state which provider and model served the run and why, and declared policies (for example, privacy class, provider restrictions, budget, or citation and personal-data constraints) must be enforced, with violations recorded rather than silently retried. This reflects the inscrutability concern in AI management (Berente et al., 2021), the need for explanations tailored to auditors (Felzmann et al., 2019), and risk-management functions in Tabassi (2023).

DR4 (Data minimization). The record must not disclose secrets or personal data beyond what accountability requires, so that it can be shared with auditors without creating a new privacy risk. This reflects data protection obligations and the transparency–privacy tension discussed by Felzmann et al. (2019).

DR5 (Low adoption cost). Producing the record must add negligible runtime overhead and require little change to existing application code, because accountability mechanisms that impose high costs are bypassed in practice (Sculley et al., 2015; Amershi et al., 2019).

4.2 Design principles

We formulate four design principles (DPs), following the convention that a principle names the action, the mechanism, and the rationale (Gregor and Hevner, 2013).

DP1 (Receipt by default). Provide the runtime with a signing key so that *every* run, whether successful or failed, emits a receipt that is canonicalized, signed, and verifiable with a public key, because accountability evidence that depends on developer discipline is incomplete. (Addresses DR1, DR5.)

DP2 (Commit to content, carry hashes). Have the receipt commit to content-addressed hashes of inputs and outputs rather than to the content itself, and keep the content in a separate artifact store, because this makes receipts small and shareable while still allowing replay. (Addresses DR2, DR4, DR5.)

DP3 (Decide deterministically and record the decision). Implement provider and model selection as a pure function of a capability catalog, the request, and the policy, and record the selected route in the receipt, because a decision that can be recomputed can be audited. Treat output-policy violations as terminal, so that a run is never retried until a banned output passes. (Addresses DR3.)

Run intent (task, artifacts, outputs, policy, contract) → Deterministic router (records route) → Provider adapter (one of seven) → Output invariants (terminal on violation) → Receipt: assemble → redact → canonicalize (RFC 8785) → DSSE pre-authentication encoding → Ed25519 signature → {verify with public key; materialize and replay offline}

Figure 1: Run life cycle in Lattice. TODO(figure): replace with a vector diagram (Fig1.eps) per journal artwork guidelines

Table 3: Design principles and their instantiation in Lattice

Principle	Mechanism in Lattice
DP1 Receipt by default	A signer configured once at initialization causes a receipt on every terminal outcome. Receipts use RFC 8785 JSON canonicalization (Rundgren et al., 2020), a DSSE envelope (Secure Systems Lab, 2021), and Ed25519 signatures (Josefsson and Liusvaara, 2017; Bernstein et al., 2012). A key set records each key as active, retired, or revoked, and an explicit schema version blocks downgrade to the original unversioned schema.
DP2 Commit to content	Receipts contain SHA-256 hashes of each input artifact, a Merkle root over artifact lineage (Merkle, 1988), and a hash of the outputs; artifacts are stored by content address. Monetary cost is serialized as an I-JSON string (Bray, 2015).
DP3 Deterministic decision	The router applies hard filters (modality, context window, structured output) and policy filters (allow and deny lists, privacy class, logging, latency, budget), then ranks survivors by a score combining cost, latency, and context headroom. Ties break on provider and model identifiers, not on catalog position. Output invariants (`mustCite`, `noPII`, `fieldFromTable`, `matches`) are evaluated by a pure function; violations are terminal and excluded from fallback.
DP4 Redact, then sign; verify, then load	Canonicalization is called only on the output of redaction. Materialization for replay verifies the receipt before any artifact is loaded.

DP4 (Redact before signing, verify before loading). Apply redaction before-canonicalization and signing, so that sensitive values never enter the signed bytes, and verify a receipt before loading any referenced artifact, so that a forged receipt cannot trigger side effects. (Addresses DR1, DR2, DR4.)

5 The Artifact: Lattice

Lattice is an open-source TypeScript library and command-line tool (Turlapati, 2026b). A developer describes a run by its task, artifacts (text, structured data, images, audio, documents, or tool results), declared outputs with schemas, a policy, and an optional contract of output invariants. Figure 1 summarizes the run life cycle, and Table 3 maps principles to mechanisms.

Provider coverage. Seven provider adapters (OpenAI, generic OpenAI-compatible endpoints, Anthropic, Google Gemini, xAI, OpenRouter, and LM Studio for local models) implement a common contract so that a receipt has the same meaning regardless of which provider served the run. The router consults a capability catalog generated from a public model registry, supplemented by hand-maintained profiles.

Multi-agent runs. For agent crews, each child receipt can carry the content identifier of its parent receipt. The crew-start receipt hashes the crew plan rather than leaving the output hash empty. Completion receipts hash each agent's output. A closing receipt lists the member content identifiers, so a dropped leaf fails chain verification.

Table 4: Evaluation of design requirements: experiments, criteria (Prat et al., 2015), and outcomes

Requirement	Experiment	Criterion	Outcome
DR1 Integrity	Tamper suite on 70 receipts; chain splicing; verifier opt-ins	Robustness	Single-receipt attacks 3570/3570; replay and open-chain tails are opt-in
DR2 Reproducibility	Record, persist, reload, and replay 350 runs across 7 adapters	Consistency	Hash match 350/350; does not cover live model behavior
DR3 Decisions and policy	1000 routing requests with an independent oracle; labeled invariants	Goal attainment	No policy violations (0); permutation-stable (1.0000); PII and must-cite below
DR4 Minimization	Secret-leakage scan of signed receipts	Goal attainment	Marker secrets in signed bytes: 0 of 14 rows
DR5 Low cost	Latency over 1000 trials; receipt sizes	Efficiency	Signing delta 0.188 ms (small text) to 1.447 ms (1 MB image), p50

External executors. An organization that already uses another orchestration framework can call an audit helper that mints the same receipt family from externally produced request, response, and output data. This supports incremental adoption (DR5), at the price of trusting the external executor's account of the provider call.

Command-line tool. The tool verifies receipts, replays them offline against stored artifacts, compares receipts structurally, and gates regressions in cost and output consistency across a corpus of receipts in continuous integration. It can also export run events through OpenTelemetry (OpenTelemetry Authors, 2026), and tool results from the Model Context Protocol (Model Context Protocol, 2025) can be recorded as artifacts.

6 Evaluation

6.1 Evaluation design

We conducted an artificial, summative evaluation (Venable et al., 2016) of the instantiation against DR1–DR5. The harness is published with the artifact (`eval/run-all.sh`). All provider calls were served by in-process fakes that return provider-shaped responses, so no network, credentials, or paid services were involved. Synthetic data were generated with fixed random seeds. Hardware, the Node version, and the git commit are in Appendix A and in `eval/results/env.json`, not in this narrative, because they are properties of the measurement machine. Table 4 maps requirements to experiments. Tables in the appendix are generated from the JSON the harness writes. The prose below quotes those generated values through macros, so a number in the text is the number in the file.

6.2 Integrity under attack (DR1)

We generated 70 receipts from runs across all seven adapters and applied the mutations in Table 8. Of the cases the harness classifies as attacks, 3570 of 3570 were rejected. Body edits failed signature verification whether the attacker kept the original signature, re-canonicalized correctly, or re-signed with an untrusted key

under the legitimate key identifier. Non-canonical encodings were rejected as canonicalization mismatches. A valid receipt presented a second time is accepted: the verifier keeps no replay cache. Audience, nonce, and a caller-supplied version floor are checked only when requested. `expiresAt`, when the issuer set it, is enforced on every verification. Presenting the same receipt again is therefore not, by itself, a failed verification.

Chain checks are separate from the ten `VerifyErrorKind` values. `verifyChain` returns `VerifyChainErrorKind`: an invalid link, no root, multiple roots, an orphan link, a run-identifier mismatch, a duplicate content identifier, a count mismatch, or a root mismatch. Reordering the receipt array is not an error, because the verifier follows parent links rather than array position. Dropping a middle link, splicing a foreign link, or replacing the root fails that check. Truncating the tail of an open chain does not. The result is success plus an advisory unless the caller passes `expectedCount`. A crew chain that declares `chainClosure` and lists member content identifiers on a closing receipt fails closed when a leaf is missing, with no expected count. Table 11 records those opt-in checks.

6.3 Reproducibility (DR2)

For each of the seven adapters we recorded runs against provider-shaped fixtures, wrote the receipt and content-addressed artifacts to disk, reloaded them, verified the receipt, materialized a replay envelope, replayed offline, and recomputed the output hash. 350 of 350 runs verified, and 350 reproduced the signed output hash. Re-executing each run against the same fixture matched in 350 cases, and a one-character change to a recorded output was detected in 350 cases (Table 9). These results show that the record is internally consistent. They do not show that a live model would reproduce its output. The crew-start receipt now hashes the plan (`{kind: crew-start, rootId, childIds}`). Completion receipts hash agent output. The closing receipt hashes the member list.

6.4 Inspectable decisions and policy enforcement (DR3)

Routing. Using a synthetic catalog of 40 model capabilities and 1000 random requests with random policies, repeated decisions were identical at rate 1.0000. An independently implemented oracle found 0 selected routes that violated provider allow or deny lists, privacy class, logging, latency, or image-modality constraints, and 0 cases in which the router reported no route although a feasible candidate existed. Reordering the catalog changed the selected model at a same-selection rate of 1.0000. Catalog position is not part of the score. Ties break on provider and model identifiers, which are recorded with the decision. Auditors still need the catalog version, because the candidates themselves come from that catalog.

Output invariants. We built a labeled set of synthetic strings containing in-scope personal data (email addresses, United States social security numbers in dashed, spaced, and undashed forms, payment card numbers, and United States phone numbers including a plain 10-digit form), out-of-scope personal data (names with addresses, international bank account numbers, IP addresses, dates of birth), and deliberately difficult negatives such as order identifiers and stock-keeping units shaped like social security or phone numbers, and 16-digit references. On in-scope types plus those negatives, `noPII` reached precision 1 and

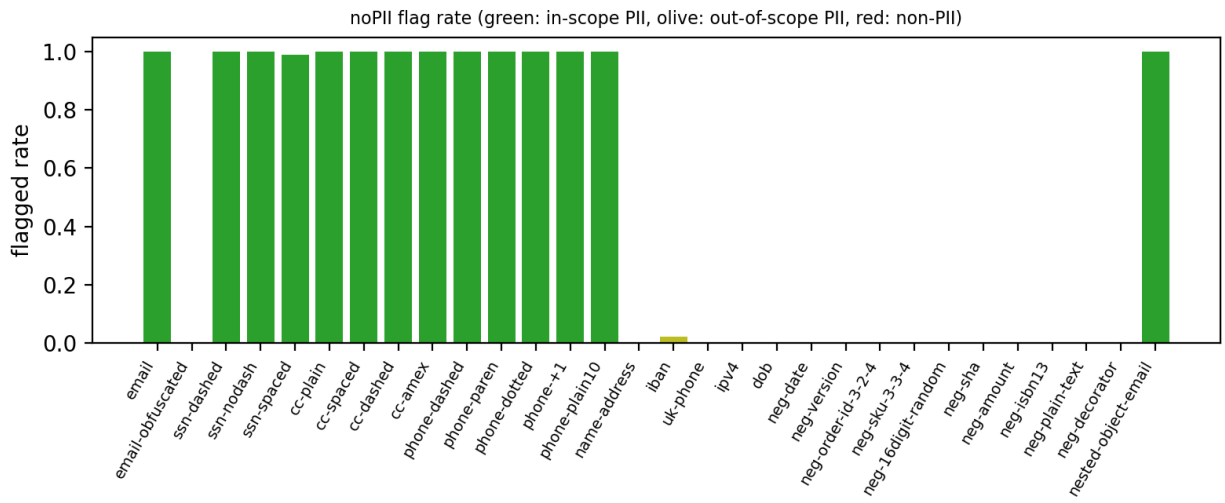


Figure 2: Share of samples flagged by the `noPII` invariant, by label. Green: in-scope personal data. Olive: out-of-scope personal data. Red: non-personal data. Plotted from `eval/results/policy.json`

recall 0.9993 ($n = 2400$, false positives 0, false negatives 1). Counting every personal-data label, including types the detectors do not claim, precision was 1 and recall 0.7005 (Table 10, Figure 2). A dashed or undashed social security number is skipped when an identifier cue such as order, invoice, sku, or ref is among the two tokens before the number. That rule is what keeps order identifiers and phone-shaped SKUs from becoming false positives, and it is also why a spaced social security number can be missed when random prose happens to contain such a cue. Because the negatives were chosen adversarially, these figures are not prevalence-weighted estimates.

The `mustCite` invariant agreed with the labels in 15 of 15 constructed cases. An inline `[name]` or `(name)` marker counts, as does a nested citations array. A citations-list entry that names the artifact is rejected when the prose's square-bracket citations name a different work. Default matching is case-sensitive and reads a string or a `source` field. Case folding and other object keys are explicit options, and the labeled set treats the default as a miss. A citations array that names the artifact, with no contradictory bracket in the prose, is accepted. The check does not judge whether the cited artifact supports the claim (Lewis et al., 2020).

6.5 Data minimization (DR4)

We placed marker secrets in the task, an input artifact, the provider credential, and the model output, then scanned the decoded signed payload, with and without a `noPII` invariant. Marker hits in signed bytes were 0 of 14 rows (Table 10 does not repeat the per-adapter log; the raw rows are in `eval/results/policy.json`). Receipts carry hashes rather than the task text or the credential. When the invariant fires, the in-process tripwire result still includes the matched substring so the caller can act on it. The value that is signed is a detector name, a length, and a mask. The earlier defect, in which the signed evidence contained the raw substring while the redaction manifest claimed otherwise, does not reproduce on this revision.

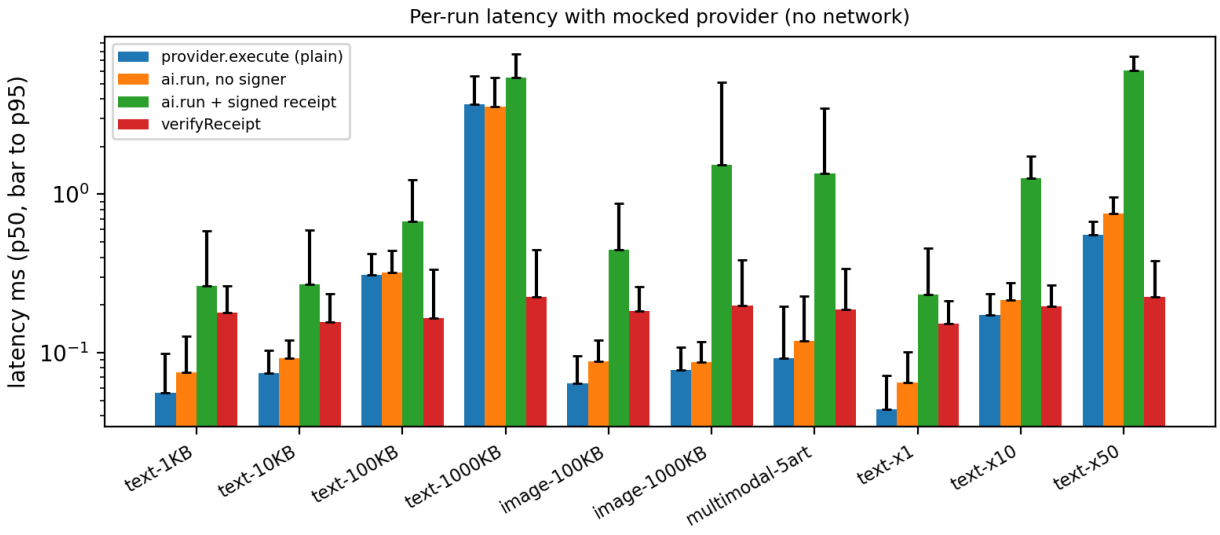


Figure 3: Per-run latency with a mocked provider across input configurations (log scale; bars show the median, whiskers extend to the 95th percentile)

6.6 Adoption cost (DR5)

Table 5 and Figure 3 report latency over 1000 trials after warm-up, against a mocked provider. Signing added 0.188 ms at the median for a 1 KB text input and 1.447 ms for a 1 MB image (signed run minus unsigned run). Verification of the small-text receipt was 0.180 ms at the median. The receipt grows with the number of input hashes rather than with payload size (Table 6). Linked receipts and chain verification scale with depth (Table 7, Figure 4). Because the provider was a zero-latency fake, these figures are the runtime's own cost, not a share of a hosted model's latency. We did not measure real provider latency.

6.7 Planned naturalistic evaluation with developers and auditors

The artificial evaluation establishes technical efficacy but not utility for the people who must act on the evidence. Following FEDS (Venable et al., 2016), we plan a naturalistic episode. [TODO: Lakshman and Prof. Neouchi: approve, adapt, or replace the following design, and obtain UT Dallas IRB determination before recruiting.]

- *Participants:* 12–20 participants in two roles, software developers who build LLM features and auditors or compliance analysts.
- *Tasks:* (a) developers instrument an existing LLM feature with receipts, measuring time to adoption and code changes (DR5); (b) auditors receive a set of runs, some tampered and some with policy violations, as either conventional logs or receipts with the command-line tool, and must identify which runs are trustworthy and which violated policy (DR1, DR3).
- *Design:* within-subject comparison of logs and receipts with counterbalanced order.

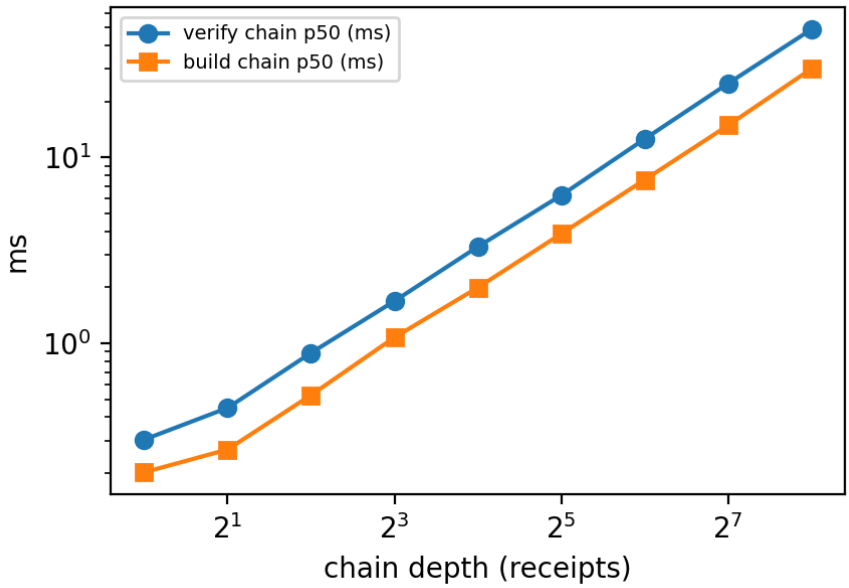


Figure 4: Build and verification time of linked receipt chains by depth

- *Measures:* task accuracy and time, perceived usefulness and ease of use, and semi-structured interview data on fit with audit procedures and frameworks such as ISO/IEC 42001 (ISO/IEC, 2023).
- *Analysis:* paired tests for accuracy and time; thematic analysis of interviews to refine the design principles.

7 Discussion

7.1 Contribution to IS knowledge

In terms of the knowledge contribution framework of Gregor and Hevner (2013), this study offers an improvement: a new solution to a known problem, auditable record keeping, for a new application context. Its contribution has two levels. At the level of nascent design theory (Gregor and Jones, 2007), it contributes five requirements and four principles for run-level AI accountability that apply beyond the specific artifact. At the level of the instantiation, it contributes an open-source artifact and a reusable evaluation harness, together with quantitative evidence about which principles the instantiation realizes. The evaluation also yields design knowledge that is easy to overlook. A self-attested receipt guarantees integrity of what was signed, not freshness, unless the issuer set an expiry or the verifier asks for an audience, a nonce, or a maximum age. Linking receipts does not by itself reveal a missing tail. A closing receipt that lists members does. A redaction manifest is a claim unless the signed bytes are the masked form, which this revision measures. These lessons transfer to any organization that designs audit trails for AI-enabled processes.

The study also connects IS research on AI governance (Berente et al., 2021; Mäntymäki et al., 2022; Papagiannidis et al., 2025) with the security literature on tamper-evident records (Schneier and Kelsey, 1999;

Crosby and Wallach, 2009; Laurie et al., 2013). Governance research has described *what* organizations must be able to show. This study demonstrates *how* an information system can be designed so that the evidence is produced as a by-product of normal operation rather than as a separate compliance activity, extending the idea of sociotechnical envelopment (Asatiani et al., 2021) to the evidentiary layer.

7.2 Implications for practice

For managers and compliance officers, the findings suggest that run-level evidence can be produced at negligible runtime cost, which removes a common argument against comprehensive logging of AI use. Evidence of this kind maps onto record-keeping duties in the EU AI Act (European Parliament and Council of the European Union, 2024), onto the measurement and management functions of the NIST AI RMF (Tabassi, 2023), and onto the documented controls an ISO/IEC 42001 management system expects (ISO/IEC, 2023). It can supply concrete artifacts for the application layer of LLM audits (Mökander et al., 2023). Three cautions follow from the evaluation. First, an issuer-signed receipt proves integrity, not honesty. Organizations that need protection against a dishonest operator should combine receipts with receiver-side attestations or transparency logs, as in Sello (Figuera, 2026) or Certificate Transparency (Laurie et al., 2013). Second, auditors should insist on chain verification and on the catalog version used for routing. Third, simple pattern-based personal-data checks should not be treated as a complete privacy control. On the in-scope labeled set they were precise, and they still miss types they do not claim, plus the occasional number that sits next to an identifier cue.

For developers, the design principles suggest a pattern: configure signing once, commit to hashes rather than content, make consequential decisions pure functions that can be recomputed, and verify before loading.

7.3 Limitations and future research

The evaluation has several limitations, which are also tracked publicly (Turlapati, 2026a).

- *Mocked providers.* All provider calls were simulated, so overhead relative to real latency, live replay drift, and provider-specific behavior were not measured.
- *Replay binding is opt-in.* Audience, nonce, and a raised version floor are not required for verification. A receipt with no `expiresAt` does not expire unless the caller passes `maxAgeMs`. There is no replay cache. Advisories warn. They do not reject.
- *Open chains can still be tail-truncated.* Without `expectedCount` or a closing member list, truncation is an advisory. Crews minted by this revision do write that list.
- *Signer failure is visible, not fail-closed by default.* If the signer throws, the run still succeeds and the result carries `receiptError` unless the caller selects throw-on-signer-error. A run with no signer emits no receipt.
- *Regex personal-data checks.* Detectors cover email, United States social security numbers, payment cards with a known issuer prefix, and United States phone numbers. Names, addresses, and non-

United States formats are out of scope. An identifier cue suppresses lookalikes and can also suppress a real number.

- *must-cite is syntactic.* It does not decide whether a cited artifact supports the prose.
- *Self-attestation and synthetic data.* Receipts are signed by the issuing runtime. The policy evaluation used synthetic data.
- *No human evaluation yet* (Sect. 6.7).

Future research can combine issuer receipts with receiver attestations and transparency logs, study how auditors use receipts in practice, and examine how receipts can serve as evidence in conformity assessments under emerging AI regulation.

8 Conclusion

AI-enabled information systems need records that auditors can verify and operators can reproduce. Following design science research, this study derived requirements and principles for run-level AI accountability, instantiated them in Lattice, and evaluated the instantiation offline. Where the harness required rejection, single-receipt tamper attempts were rejected, recorded runs replayed to the signed output hash, routing policies were not violated, and signing added milliseconds per run (Table 4). Freshness binding and open-chain length remain opt-in. Personal data matched by the privacy check is masked before signing. Together, the measured results and the remaining opt-ins are design knowledge for organizations that want AI evidence produced during the run.

Declarations

Funding [TODO: State funding, or "No funding was received for conducting this study."]

Competing interests [TODO: Lakshman Turlapati is the author and maintainer of Lattice and is affiliated with Full Self Browsing, which publishes the artifact; declare as applicable.]

Data and code availability The artifact and the evaluation harness, including raw results, are available at `https://github.com/fullselfbrowsing/Lattice`. [TODO: Push the evaluation branch and archive a tagged release with a DOI (e.g., Zenodo) before submission.]

Author contributions [TODO: CRediT statement, e.g., Conceptualization: L.T., R.N.; Software: L.T.; Methodology: L.T., R.N.; Writing: L.T.; Supervision: R.N.]

Ethics approval Not applicable to the reported evaluation, which involved no human participants. [TODO: Required for the planned study in Sect. 6.7.]

Table 5: Per-run latency (ms) with a mocked OpenAI-shaped provider, 1000 trials after 100 warm-up. Overhead = signed run − unsigned run.

Config	plain p50	run p50	signed p50	signed p95	signed p99	verify p50
text-1KB	0.056	0.075	0.263	0.590	0.826	0.180
text-10KB	0.074	0.092	0.271	0.591	0.804	0.157
text-100KB	0.308	0.320	0.672	1.238	1.545	0.165
text-1000KB	3.687	3.589	5.440	7.665	8.823	0.226
image-100KB	0.064	0.088	0.445	0.877	4.850	0.182
image-1000KB	0.078	0.087	1.534	5.076	6.479	0.199
multimodal-5art	0.092	0.118	1.356	3.508	7.700	0.187
text-x1	0.044	0.065	0.233	0.456	0.827	0.153
text-x10	0.172	0.215	1.266	1.746	2.553	0.197
text-x50	0.552	0.758	6.020	7.387	10.009	0.225

Isolated: createReceipt p50 0.059 / p99 0.223; verifyReceipt p50 0.125 / p99 0.232; receiptCid p50 0.0508

Table 6: Receipt envelope size vs. input payload.

Config	payload B	artifacts	receipt B
text-1KB	1024	1	1183
text-10KB	10240	1	1183
text-100KB	102400	1	1183
text-1000KB	1024000	1	1183
image-100KB	102400	2	1271
image-1000KB	1024000	2	1271
multimodal-5art	618496	5	1539
text-x1	1024	1	1183
text-x10	10240	10	1987
text-x50	51200	50	5559

A Reproducibility

The narrative quotes the harness through macros generated by `eval/scripts/06-report.py`. This run used Node v24.22.1, Intel(R) Xeon(R) Processor (4 cores, 15.6 GB), platform linux 6.12.94+, Lattice 1.6.0, commit `45493340b1c26c0e30307c558bea6a0bd17f7ec8`, at 2026-10-21T16:29:36.012Z. The library tests passed 1316 of 1316 (0 failed). The command-line tests passed 177 of 177 (0 failed). Those counts document that the suite ran. They are not the argument for the design.

Provider calls were mocked. No API credential was read from the environment. Regenerate with `eval/run-all.sh` and `pnpm bench:receipts`. The repository benchmark, which is separate from the harness tamper suite, writes `paper/bench/results.json`. Springer Nature's `sn-jnl` class was not applied: the journal's template download is script-rendered and was not available as a class file in this build. The source uses the article class, `natbib`, and an author-year style close to the Information Systems Frontiers APA option. Before submission, port the body into `\documentclass[pdflatex,sn-apa]{sn-jnl}`.

References

Amershi, S., Begel, A., Bird, C., DeLine, R., Gall, H., Kamar, E., Nagappan, N., Nushi, B., and Zimmermann, T. (2019). Software engineering for machine learning: A case study. In *2019 IEEE/ACM 41st*

Table 7: Chained receipts (parentReceiptCid): size and verification cost by depth (30 trials).

Depth	total B	B/link	build p50 ms	verify-all p50 ms
1	1023	1023	0.202	0.303
2	2170	1085	0.268	0.449
4	4464	1116	0.526	0.888
8	9052	1132	1.076	1.688
16	18228	1139	1.996	3.312
32	36580	1143	3.889	6.266
64	73284	1145	7.595	12.599
128	146692	1146	14.913	25.074
256	293508	1147	30.116	49.059

International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), pages 291–300. IEEE.

Arslan, M. (2026). Janus: Evidence-before-effect sagas and offline-verifiable provenance for agentic LLMs. arXiv preprint arXiv:2609.38266.

Asatiani, A., Malo, P., Nagbøl, P. R., Penttinen, E., Rinta-Kahila, T., and Salovaara, A. (2021). Sociotechnical envelopment of artificial intelligence: An approach to organizational deployment of inscrutable artificial intelligence systems. *Journal of the Association for Information Systems*, 22(2):325–352.

Baskerville, R., Baiyere, A., Gregor, S., Hevner, A., and Rossi, M. (2018). Design science research contributions: Finding a balance between artifact and theory. *Journal of the Association for Information Systems*, 19(5):358–376.

Berente, N., Gu, B., Recker, J., and Santhanam, R. (2021). Managing artificial intelligence. *MIS Quarterly*, 45(3):1433–1450.

Bernstein, D. J., Duif, N., Lange, T., Schwabe, P., and Yang, B.-Y. (2012). High-speed high-security signatures. *Journal of Cryptographic Engineering*, 2(2):77–89.

Bommasani, R., Hudson, D. A., Adeli, E., Altman, R., Arora, S., et al. (2021). On the opportunities and risks of foundation models. arXiv preprint arXiv:2108.07258.

Bray, T. (2015). *The I-JSON Message Format*.

Brundage, M., Avin, S., Wang, J., Belfield, H., Krueger, G., Hadfield, G., et al. (2020). Toward trustworthy AI development: Mechanisms for supporting verifiable claims. arXiv preprint arXiv:2004.07213.

Chen, L., Zaharia, M., and Zou, J. (2023). FrugalGPT: How to use large language models while reducing cost and improving performance. arXiv preprint arXiv:2305.05176.

Crosby, S. A. and Wallach, D. S. (2009). Efficient data structures for tamper-evident logging. In *18th USENIX Security Symposium (USENIX Security 09)*, pages 317–334. USENIX Association.

European Parliament and Council of the European Union (2024). Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (artificial intelligence act). Official Journal of the European Union, L series.

Felzmann, H., Villaronga, E. F., Lutz, C., and Tamò-Larrieux, A. (2019). Transparency you can trust: Transparency requirements for artificial intelligence between legal norms and contextual concerns. *Big Data & Society*, 6(1).

Figuera, J. (2026). Notarized agents: Receiver-attested confidential receipts for AI agent actions. arXiv preprint arXiv:2606.04193.

Gebru, T., Morgenstern, J., Vecchione, B., Vaughan, J. W., Wallach, H., III, H. D., and Crawford, K. (2021). Datasheets for datasets. *Communications of the ACM*, 64(12):86–92.

Gregor, S. and Hevner, A. R. (2013). Positioning and presenting design science research for maximum impact. *MIS Quarterly*, 37(2):337–355.

Gregor, S. and Jones, D. (2007). The anatomy of a design theory. *Journal of the Association for Information Systems*, 8(5):312–335.

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., and Fritz, M. (2023). Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, CCS '23, pages 79–90. ACM.

Haber, S. and Stornetta, W. S. (1991). How to time-stamp a digital document. *Journal of Cryptology*, 3(2):99–111.

Herschel, M., Diestelkämper, R., and Ben Lahmar, H. (2017). A survey on provenance: What for? what form? what from? *The VLDB Journal*, 26(6):881–906.

Hevner, A. R., March, S. T., Park, J., and Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1):75–106.

ISO/IEC (2023). ISO/IEC 42001:2023 information technology – artificial intelligence – management system. International Organization for Standardization.

Josefsson, S. and Liusvaara, I. (2017). *Edwards-Curve Digital Signature Algorithm (EdDSA)*.

Kang, D., Hashimoto, T., Stoica, I., and Sun, Y. (2022). Scaling up trustless DNN inference with zero-knowledge proofs. arXiv preprint arXiv:2210.08674.

Kapoor, S. and Narayanan, A. (2023). Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9):100804.

Laurie, B., Langley, A., and Kasper, E. (2013). *Certificate Transparency*.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. arXiv preprint arXiv:2005.11401 (NeurIPS 2020).

Li, M. (2025). From cloud-native to trust-native: A protocol for verifiable multi-agent systems. arXiv preprint arXiv:2507.22077.

Merkle, R. C. (1988). *A Digital Signature Based on a Conventional Encryption Function*, pages 369–378. Springer Berlin Heidelberg.

Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., and Gebru, T. (2019). Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, FAT* '19, pages 220–229. ACM.

Model Context Protocol (2025). Model context protocol specification (2025-06-18).

Moreau, L., Clifford, B., Freire, J., Futrelle, J., Gil, Y., Groth, P., Kwasnikowska, N., Miles, S., Missier, P., Myers, J., Plale, B., Simmhan, Y., Stephan, E., and den Bussche, J. V. (2011). The open provenance model core specification (v1.1). *Future Generation Computer Systems*, 27(6):743–756.

Mäntymäki, M., Minkkinen, M., Birkstedt, T., and Viljanen, M. (2022). Defining organizational ai governance. *AI and Ethics*, 2(4):603–609.

Mökander, J. and Floridi, L. (2021). Ethics-based auditing to develop trustworthy ai. *Minds and Machines*, 31(2):323–327.

Mökander, J., Schuett, J., Kirk, H. R., and Floridi, L. (2023). Auditing large language models: a three-layered approach. *AI and Ethics*, 4(4):1085–1115.

Newman, Z., Meyers, J. S., and Torres-Arias, S. (2022). Sigstore: Software signing for everybody. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, CCS '22, pages 2353–2367. ACM.

Novelli, C., Taddeo, M., and Floridi, L. (2023). Accountability in artificial intelligence: what it is and how it works. *AI & SOCIETY*, 39(4):1871–1882.

Ong, I., Almahairi, A., Wu, V., Chiang, W.-L., Wu, T., Gonzalez, J. E., et al. (2024). RouteLLM: Learning to route LLMs with preference data. arXiv preprint arXiv:2406.18665.

OpenSSF (2023). SLSA specification v1.0: Supply-chain levels for software artifacts.

OpenTelemetry Authors (2026). Opentelemetry specification.

Papagiannidis, E., Mikalef, P., and Conboy, K. (2025). Responsible artificial intelligence governance: A review and research framework. *The Journal of Strategic Information Systems*, 34(2):101885.

Peffers, K., Tuunanen, T., Rothenberger, M. A., and Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3):45–77.

Prat, N., Comyn-Wattiau, I., and Akoka, J. (2015). A taxonomy of evaluation methods for information systems artifacts. *Journal of Management Information Systems*, 32(3):229–267.

Rai, A. (2019). Explainable ai: from black box to glass box. *Journal of the Academy of Marketing Science*, 48(1):137–141.

Raji, I. D., Smart, A., White, R. N., Mitchell, M., Gebru, T., Hutchinson, B., Smith-Loud, J., Theron, D., and Barnes, P. (2020). Closing the ai accountability gap: defining an end-to-end framework for internal algorithmic auditing. In *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, FAT* '20, pages 33–44. ACM.

Rundgren, A., Jordan, B., and Erdtman, S. (2020). *JSON Canonicalization Scheme (JCS)*.

Samuel, J., Mathewson, N., Cappos, J., and Dingledine, R. (2010). Survivable key compromise in software update systems. In *Proceedings of the 17th ACM conference on Computer and communications security*, CCS '10, pages 61–72. ACM.

Schneider, J., Abraham, R., Meske, C., and Vom Brocke, J. (2022). Artificial intelligence governance for businesses. *Information Systems Management*, 40(3):229–249.

Schneier, B. and Kelsey, J. (1999). Secure audit logs to support computer forensics. *ACM Transactions on Information and System Security*, 2(2):159–176.

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems 28*.

Secure Systems Lab (2021). DSSE: Dead simple signing envelope (specification). GitHub repository.

Shneiderman, B. (2020). Bridging the gap between ethics and practice: Guidelines for reliable, safe, and trustworthy human-centered ai systems. *ACM Transactions on Interactive Intelligent Systems*, 10(4):1–31.

Sonnenberg, C. and vom Brocke, J. (2012). *Evaluations in the Science of the Artificial – Reconsidering the Build-Evaluate Pattern in Design Science Research*, pages 381–397. Springer Berlin Heidelberg.

Sun, H., Li, J., and Zhang, H. (2024). zkllm: Zero knowledge proofs for large language models. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, pages 4405–4419. ACM.

Tabassi, E. (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*.

Torres-Arias, S., Afzali, H., Kuppusamy, T. K., Curtmola, R., and Cappos, J. (2019). in-toto: Providing farm-to-table guarantees for bits and bytes. In *28th USENIX Security Symposium (USENIX Security 19)*, pages 1393–1410. USENIX Association.

Turlapati, L. (2026a). Eval harness findings: matched PII signed into no-PII receipts, receipt replay and chain gaps, and 8 more issues (blocks paper submission).

Turlapati, L. (2026b). Lattice: Capability runtime SDK for multimodal AI applications (source code). GitHub repository.

Venable, J., Pries-Heje, J., and Baskerville, R. (2016). Feds: a framework for evaluation in design science research. *European Journal of Information Systems*, 25(1):77–89.

vom Brocke, J., Hevner, A., and Maedche, A. (2020). *Introduction to Design Science Research*, pages 1–13. Springer International Publishing.

Walls, J. G., Widmeyer, G. R., and El Sawy, O. A. (1992). Building an information system design theory for vigilant eis. *Information Systems Research*, 3(1):36–59.

Wieringa, M. (2020). What to account for when accounting for algorithms: a systematic literature review on algorithmic accountability. In *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, FAT* '20, pages 1–18. ACM.

Table 8: Tamper suite over 70 real receipts: rejection rate per mutation and error kind.

Class	Mutation	Rejected	Kind
body-edit-keep-sig	edit-outputHash	70/70	signature-invalid:70
body-edit-jcs-keep-sig	edit-outputHash	70/70	signature-invalid:70
body-edit-attacker-sig-legit-kid	edit-outputHash	70/70	signature-invalid:70
body-edit-keep-sig	edit-inputHashes	70/70	signature-invalid:70
body-edit-jcs-keep-sig	edit-inputHashes	70/70	signature-invalid:70
body-edit-attacker-sig-legit-kid	edit-inputHashes	70/70	signature-invalid:70
body-edit-keep-sig	edit-route-provider	70/70	signature-invalid:70
body-edit-jcs-keep-sig	edit-route-provider	70/70	signature-invalid:70
body-edit-attacker-sig-legit-kid	edit-route-provider	70/70	signature-invalid:70
body-edit-keep-sig	edit-model	70/70	signature-invalid:70
body-edit-jcs-keep-sig	edit-model	70/70	signature-invalid:70
body-edit-attacker-sig-legit-kid	edit-model	70/70	signature-invalid:70
body-edit-keep-sig	edit-usage-cost	70/70	signature-invalid:70
body-edit-jcs-keep-sig	edit-usage-cost	70/70	signature-invalid:70
body-edit-attacker-sig-legit-kid	edit-usage-cost	70/70	signature-invalid:70
body-edit-keep-sig	edit-verdict	70/70	signature-invalid:70
body-edit-jcs-keep-sig	edit-verdict	70/70	signature-invalid:70
body-edit-attacker-sig-legit-kid	edit-verdict	70/70	signature-invalid:70
body-edit-keep-sig	edit-issuedAt	70/70	signature-invalid:70
body-edit-jcs-keep-sig	edit-issuedAt	70/70	signature-invalid:70
body-edit-attacker-sig-legit-kid	edit-issuedAt	70/70	signature-invalid:70
body-edit-keep-sig	edit-runId	70/70	signature-invalid:70
body-edit-jcs-keep-sig	edit-runId	70/70	signature-invalid:70
body-edit-attacker-sig-legit-kid	edit-runId	70/70	signature-invalid:70
body-edit-keep-sig	drop-redactions	70/70	version-mismatch:70
body-edit-jcs-keep-sig	drop-redactions	70/70	version-mismatch:70
body-edit-attacker-sig-legit-kid	drop-redactions	70/70	version-mismatch:70
body-edit-keep-sig	edit-redactionPolicy	70/70	signature-invalid:70
body-edit-jcs-keep-sig	edit-redactionPolicy	70/70	signature-invalid:70
body-edit-attacker-sig-legit-kid	edit-redactionPolicy	70/70	signature-invalid:70
body-edit-keep-sig	add-field	70/70	canonicalization-mismatch:70
body-edit-jcs-keep-sig	add-field	70/70	signature-invalid:70
body-edit-attacker-sig-legit-kid	add-field	70/70	signature-invalid:70
body-edit-keep-sig	unicode-nfd	70/70	signature-invalid:70
body-edit-jcs-keep-sig	unicode-nfd	70/70	signature-invalid:70
body-edit-attacker-sig-legit-kid	unicode-nfd	70/70	signature-invalid:70
encoding	key-reorder	70/70	canonicalization-mismatch:70
encoding	whitespace	70/70	canonicalization-mismatch:70
signature	sig-bitflip	70/70	signature-invalid:70
signature	sig-swap-same-key	70/70	signature-invalid:70
signature	sig-truncate	70/70	signature-invalid:70
signature	sig-empty-list	70/70	envelope-malformed:70
signature	payload-truncate	70/70	envelope-malformed:70
signature	payloadType-change	70/70	envelope-malformed:70
key	keyid-to-other-trusted	70/70	signature-invalid:70
key	keyid-unknown	70/70	key-not-found:70
key	forge-attacker-key-own-kid	70/70	key-not-found:70
key	resign-revoked-key	70/70	key-revoked:70
key	resign-other-key-body-kid-mismatch	70/70	signature-invalid:70
downgrade	version-v1-resigned-valid-key	70/70	schema-version-too-low:70
downgrade	version-absent-resigned-valid-key	70/70	schema-version-too-low:70
control-accept	untouched	0/70	accepted(active):70
control-accept	retired-key-legit-signature	0/70	accepted(retired):70
control-accept	version-v1.1-resigned-valid-key (key holder)	0/70	accepted(active):70
replay	same-receipt-presented-again (no nonce/audience)	0/70	accepted(active):70
Chain attacks (50 chains of 8): per-receipt verify vs. harness chain verifier			
chain	swap-adjacent-links	0.00 / 0.00	tail truncation of leaf receipts is not checked; pass expectedCount
chain	drop-middle-link	0.00 / 1.00	orphan-link
chain	splice-foreign-link	0.00 / 1.00	orphan-link
chain	truncate-tail	0.00 / 0.00	tail truncation of leaf receipts is not checked; pass expectedCount
chain	truncate-tail-expectedCount	0.00 / 1.00	count-mismatch
chain	replace-root	0.00 / 1.00	orphan-link

Table 9: Offline replay over recorded fixtures, per adapter.

Adapter	runs	verified	hash match	live rerun match	altered detected
openai	50	50	50	50	50
openai-compatible	50	50	50	50	50
anthropic	50	50	50	50	50
gemini	50	50	50	50	50
xai	50	50	50	50	50
openrouter	50	50	50	50	50
lm-studio	50	50	50	50	50

Table 10: Policy checks on a synthetic labeled set (100 samples per label).

Set	n	precision	recall	FP
noPII, in-scope PII types	2400	1	0.9993	0
noPII, all PII types	3000	1	0.7005	0
noPII, out-of-scope types	1600	1	0.0033	0
mustCite agreement	15	15/15 cases agree with label		
routing determinism	20000	identical 1.0000; permuted-catalog same 1.000; violations 0		

Table 11: Verifier opt-ins, chain closure, and signer-failure modes.

Check	Result
plain-verifies	pass ()
advisories-on-unbound-receipt	pass (3)
bound-accepts-matching-options	pass ()
audience-mismatch	pass (audience-mismatch)
nonce-mismatch	pass (nonce-mismatch)
expired	pass (receipt-expired)
minVersion-v1.4-accepts-current	pass ()
minVersion-floor-rejects-v1.1	pass (schema-version-too-low)
v1.1-still-verifies-without-minVersion	pass ()
truncate-without-expectedCount	pass (accepted)
truncate-with-expectedCount	pass (count-mismatch)
closed-chain-accepts	pass ()
closed-chain-dropped-leaf	pass (count-mismatch)
signer-flag-default	pass (signer-failed)
signer-throw	pass (ReceiptSigningError)
no-signer-no-receipt	pass ()